Predicting Behaviour Trajectories with LSTM and GRU

The theory behind `w2_trajectory_models.ipynb`

Workshop 2 · Group 2

Abstract

This document collects the theory behind the trajectory notebook of Workshop 2. The goal is no longer to classify whether a user will churn, but to predict what the user will do over the next 7 days: whether they listen, how much they listen, whether they pay, how much they pay and whether they cancel. The model is then used as a simulator for the retention agents in `wfinal`. The document explains how the 90-day window is split, what is predicted and why, the recurrent architecture, the composite loss, how full weeks are simulated by predicting events and rebuilding the subscription state, and how everything is evaluated against naive predictors.

Contents

1 From classification to trajectories **2**

1.1 What changes . . . 2

1.2 What it is for . . . 2

1.3 Splitting the window . . . 2

1.4 Why $H = 7$ . . . 3

2 The data **3**

2.1 The 20 channels . . . 3

2.2 Export encoding . . . 3

2.3 Masked standard scaling . . . 4

2.4 The 13 targets . . . 4

3 Events and states **5**

3.1 States are arithmetic on events . . . 5

3.2 Predict events, rebuild states . . . 5

3.3 Fit with the environment . . . 5

4 The model **5**

4.1 Recurrent encoder . . . 5
4.2 User summary and normalisation . . . 6
4.3 Direct multi-horizon head . . . 6
4.4 Capacity . . . 7

5 Hurdle model for listening **7**
5.1 The problem of zeros . . . 7
5.2 Two-part decomposition . . . 8

6 The composite loss **8**
6.1 Binary cross-entropy . . . 8
6.2 Huber loss . . . 8
6.3 Masked averages . . . 9
6.4 The five terms . . . 9
6.5 Fixed weights from a constant predictor . . . 9
6.6 Why there is no `pos_weight` on payments . . . 10

7 Training and hyper-parameter search **10**
7.1 Backpropagation through time . . . 10
7.2 Optimisation . . . 10
7.3 Partitions . . . 11
7.4 Optuna . . . 11

8 Simulation: from probabilities to one possible week **11**
8.1 Sampling, not thresholding . . . 11
8.2 State reconstruction rules . . . 12
8.3 Chaining weeks . . . 12
8.4 Use from the agent's environment . . . 13

9 Evaluation **13**
9.1 Principle: compare with naive predictors . . . 13
9.2 Listening . . . 13
9.3 Events: activity and payment . . . 14
9.4 Amount and plan . . . 15
9.5 Realism of the simulation . . . 15

9.6 Error of the rebuilt state 15
9.7 LSTM versus GRU: paired bootstrap 16

10 Limitations **16**

1 From classification to trajectories

1.1 What changes

In the classification notebook (`w2_neural_models.ipynb`), each user i is represented by a matrix $X_i \in \mathbb{R}^{90\times 20}$ (90 days, 20 channels) and the model outputs a single number: the churn probability $\hat{p}_i = P(y_i = 1 \mid X_i)$. It is a *many-to-one* problem: many input steps and one output.

In the trajectory notebook the output is no longer a label but *the user's own future*: a matrix of days by variables. The problem becomes *many-to-many* and, above all, it is no longer pure classification. Some variables are continuous quantities (seconds listened, amount paid) and others are binary events (did they listen?, did they pay?). As shown below, each type of output therefore needs its own activation and its own loss.

1.2 What it is for

The agents in `wfinal` choose retention actions (reminders, discounts, downgrades, etc.). To train them, an *environment* must answer the question: "given what this user has done up to today, how will they behave over the next days?". A trajectory predictor is exactly that: a simulator of the user learned from data.

1.3 Splitting the window

The export does not need to be rebuilt. The existing 90-day window is split in two:

$$\underbrace{x_{i,0},\, x_{i,1},\, \ldots,\, x_{i,82}}_{\text{input: } T_{\text{in}}=83 \text{ days}} \;\Big|\; \underbrace{x_{i,83},\, \ldots,\, x_{i,89}}_{\text{target: } H=7 \text{ days}}. \tag{1}$$

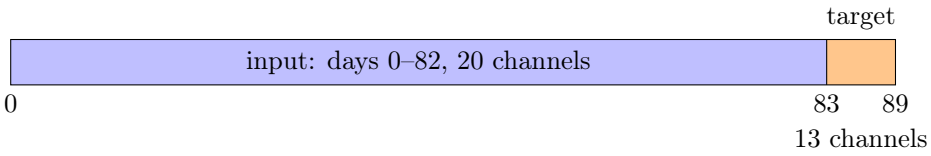


Figure 1: The 90-day window split into input and target.

Cohort	Input	Target
`train_feb`	2016-11-03 to 2017-01-24	2017-01-25 to 2017-01-31
`test` (March)	2016-12-01 to 2017-02-21	2017-02-22 to 2017-02-28

Table 1: Input and target dates of each cohort.

The target days are "future" relative to what the model sees, not relative to the calendar. This is not a problem because the model never receives the date: it only sees a sequence of 83

days and learns to continue it. As a result, in the environment it can be given the last 83 days before the cutoff and it predicts the following week (Section 8.4).

1.4 Why $H = 7$

- One week covers a full listening cycle (weekdays and weekend).
- The input stays long: 83 days hold almost three months of history.
- Daily listening is very noisy. With a longer horizon, almost all of the error would be unpredictable noise and the input would shrink.
- One agent turn maps naturally onto one week: action $\rightarrow$ 7 simulated days $\rightarrow$ new state.

2 The data

2.1 The 20 channels

Each day t of user i is a vector $x_{i,t} \in \mathbb{R}^{20}$ with three blocks:

- **Listening (0–7).** Seven counters (`num_25`, `num_50`, `num_75`, `num_985`, `num_100`, `num_unq`, `total_secs`) and the `active` flag, which is 1 if a listening record exists for that day.
- **Payment events (8–12).** `tx_count`, `tx_paid_sum`, `tx_plan_days_sum`, `tx_cancel_count`, `tx_auto_renew_count`. They are non-zero only on days with at least one payment.
- **Subscription state (13–19).** `tx_state_known`, `tx_last_is_cancel`, `tx_last_is_auto_renew`, `tx_days_since_last`, `tx_days_to_expire`, `tx_expiry_known`, `tx_state_ambiguous`. They describe the subscription at the end of each day and are carried forward until the next payment.

2.2 Export encoding

Quantities have very long tails (a user may listen for 100 seconds or for 50,000 seconds in a day). To compress them, the export stores

$$\log(1 + v) \qquad \text{for non-negative quantities,} \tag{2}$$

$$\operatorname{slog}(v) = \operatorname{sign}(v)\, \log(1 + |v|) \qquad \text{for quantities that can be negative.} \tag{3}$$

The signed logarithm is used for `tx_paid_sum` (negative amounts exist) and `tx_days_to_expire` (negative once the membership has expired). Both transformations are invertible:

$$v = e^u - 1 = \operatorname{expm1}(u), \qquad v = \operatorname{sign}(u)\left(e^{|u|} - 1\right). \tag{4}$$

All evaluation is done in this encoding, or after undoing it, never in the scaled space.

2.3 Masked standard scaling

Before entering the network, each numeric channel is clipped at an upper quantile and standardised:

$$\tilde{x}_{i,t,k} = \begin{cases} \dfrac{\min(x_{i,t,k},\, u_k) - \mu_k}{\sigma_k} & \text{if the channel is defined on that day,} \\ 0 & \text{otherwise.} \end{cases} \tag{5}$$

The mean μ_k and standard deviation σ_k are computed *only on the days where the channel is defined*: active days for listening, payment days for events and days with a known state for states. The parameters are fitted on `fit` during the search and on all of February (`development`) for the final model. Binary flags are not scaled.

> **Key idea.** After scaling, an inactive day is 0 and an *average* active day is also 0. In the scaled space, "did not listen" and "listened as usual" are the same number. This coincidence is the underlying reason for the hurdle model of Section 5.

2.4 The 13 targets

For each user the target is a tensor $Y_i \in \mathbb{R}^{7\times 13}$:

Pos.	Target	What it is	Defined on
0–6	listening	the 7 counters, scaled	active days
7	`active`	$\mathbf{1}$[listening record exists]	every day
8	`pay`	$\mathbf{1}$[`tx_count` > 0]	every day
9–10	amount and plan	`tx_paid_sum`, `tx_plan_days_sum`, scaled	payment days
11	`cancel`	$\mathbf{1}$[`tx_cancel_count` > 0]	payment days
12	`auto_renew`	$\mathbf{1}$[`tx_auto_renew_count` > 0]	payment days

Table 2: Layout of the target $Y_i \in \mathbb{R}^{7\times 13}$.

Binary targets are read from the *unscaled* tensor. For example, a day with one payment has `tx_count` $= \log 2 \approx 0.693$ in the export. With the mean and standard deviation of `fit` ($\mu \approx 0.697$, $\sigma \approx 0.037$) this becomes $(0.693 - 0.697)/0.037 \approx -0.1$. Reading the scaled value would encode "paid" as -0.1 instead of 1.

The 7 state channels are *not* targets. Section 3 explains why.

3 Events and states

3.1 States are arithmetic on events

Once the events of a day are known, the state of the next day is fully determined:

- With a payment: `days_since_last` goes back to 0, the flags take the values of the payment and the expiry moves.
- Without a payment: `days_since_last` increases by 1 and `days_to_expire` decreases by 1.

The state carries no new information: it is a deterministic function of the events and the previous state.

3.2 Predict events, rebuild states

The network therefore predicts only the 13 event targets. To simulate a week, one concrete scenario is *sampled* from the predicted probabilities (the user pays or does not pay), and the state is computed with the same rules as the export (Section 8). Two properties follow:

- **Coherence by construction.** A simulated payment always resets `days_since_last` and moves the expiry; no trajectory can contradict itself.
- **Only possible values.** Every simulated day is a state that a real user could be in, never an average of several futures.

> **Key idea.** The simulator produces *one possible future* per user, not the average of the possible futures. An environment needs exactly that, because it must be able to move forward through valid states.

3.3 Fit with the environment

In `wfinal`, the customer is stored as raw data (transactions and logs) and the `SequenceBuilder` recomputes the states from the payments. Simulated events fit directly into that representation: they are payments and listening days.

4 The model

4.1 Recurrent encoder

The network reads the 83 days in order and keeps a hidden state $h_t \in \mathbb{R}^{H_d}$ that summarises everything seen up to day t. Two cells are compared.

LSTM. It uses a separate memory c_t and three gates:

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f) \qquad \text{(forget)} \tag{6}$$

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i) \qquad \text{(input)} \tag{7}$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o) \qquad \text{(output)} \tag{8}$$

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c) \tag{9}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \tag{10}$$

$$h_t = o_t \odot \tanh(c_t). \tag{11}$$

The additive update of c_t lets the gradient travel through many steps without vanishing: if $f_t \approx 1$, the memory passes almost unchanged. Each layer has four weight blocks.

GRU. It merges memory and hidden state into a single vector:

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r) \qquad \text{(reset)} \tag{12}$$

$$u_t = \sigma(W_u x_t + U_u h_{t-1} + b_u) \qquad \text{(update)} \tag{13}$$

$$n_t = \tanh\big(W_n x_t + b_n + r_t \odot (U_n h_{t-1} + d_n)\big) \tag{14}$$

$$h_t = (1 - u_t) \odot n_t + u_t \odot h_{t-1}. \tag{15}$$

When $u_t \approx 1$ it keeps its memory; when $u_t \approx 0$ it replaces it with the candidate. It uses three weight blocks instead of four, so it is smaller and faster for the same hidden size.

Stacking. Two or three layers are stacked: the first reads the 20 channels and each following layer reads the hidden states of the one below. Initial states are zero.

4.2 User summary and normalisation

After day 82 (the last input day), the top layer outputs $h_{83} \in \mathbb{R}^{H_d}$: a learned summary of the user's whole behaviour. It is normalised with `BatchNorm1d`:

$$\tilde{h} = \gamma \odot \frac{h_{83} - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta, \tag{16}$$

where μ_B and σ_B^2 are the batch mean and variance and γ, β are learned parameters. This stabilises the scale of the input to the head and makes training easier.

4.3 Direct multi-horizon head

The head is a single linear layer that emits all 7 days at once:

$$z = W\tilde{h} + b \in \mathbb{R}^{91}, \qquad Z = \text{reshape}(z) \in \mathbb{R}^{7 \times 13}. \tag{17}$$

$Z_{h,j}$ is the output for day $83+h-1$ and target j. The 6 binary positions are *logits* (the sigmoid is applied inside the loss or at prediction time) and the 7 quantity positions are scaled values.

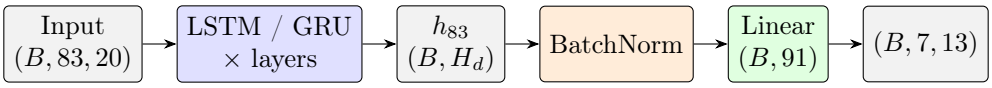


Figure 2: Architecture shared by LSTM and GRU. Only the recurrent cell changes.

Why not an autoregressive decoder? The alternative would be to predict day 83, feed it back as input, predict day 84, and so on. It has two drawbacks:

1. **Error accumulation:** each day is predicted from days that were already predicted, so errors chain together.
2. **It would need all 20 channels at every step**, including the states, to feed them back.

With a horizon of only 7 days, the direct head is simpler and sufficient. Chaining is done in blocks of one week (Section 8.3), where the states are rebuilt correctly.

4.4 Capacity

In the classifier the smallest configuration won (32 units, 2 layers), but there the network had to produce a single number. Now the vector $\tilde{h}$ must encode 91 outputs, so the search space is widened to $H_d \in \{64, 128, 256\}$.

5 Hurdle model for listening

5.1 The problem of zeros

Daily listening has a large mass at zero (days without listening) and a continuous part (days with listening). If a single regression is fitted over all days, for a user who listens with probability p and, when listening, about m (in $\log 1p$), the optimal prediction under squared loss is

$$\mathbb{E}[Y] = p \cdot m + (1-p) \cdot 0 = p\, m. \tag{18}$$

With $p = 0.5$ and $m = 9$ (about 8,000 seconds), the model predicts 4.5 (about 90 seconds) *every day*. That user does not exist: the real one listens a lot on some days and not at all on others. Moreover, since in the scaled space zero coincides with the average active day, the model can reach a low error by predicting almost zero everywhere without learning anything.

5.2 Two-part decomposition

The hurdle model separates the two questions:

$$Y = A \cdot M, \qquad A \sim \text{Bernoulli}(\pi), \qquad M \mid A = 1 \sim \text{distribution of the amount.} \tag{19}$$

- $\pi = P(\text{active})$ is learned with binary cross-entropy on `active`.
- The amount is learned with the Huber loss *on active days only.*

When a point forecast is needed (for the MAE), the model uses

$$\hat{y} = \hat{\pi} \cdot \hat{m}. \tag{20}$$

When simulating, A is sampled and, if it is 1, $\hat{m}$ is written. This yields days with and without listening, as in the real data.

The same logic applies to payments: `pay` is the hurdle, and the amount, plan, cancellation and auto-renewal are learned only on payment days. If the amount were learned on every day, since there is almost never a payment, the model would learn to predict almost zero.

6 The composite loss

6.1 Binary cross-entropy

For a binary target $y \in \{0, 1\}$ and a logit z, with $\hat{p} = \sigma(z)$:

$$\text{BCE}(z, y) = -\big[y \log \sigma(z) + (1 - y) \log(1 - \sigma(z))\big]. \tag{21}$$

Its derivative with respect to the logit is simply $\partial\,\text{BCE}\,/\partial z = \sigma(z) - y$. PyTorch computes it together with the sigmoid (`binary_cross_entropy_with_logits`), which avoids evaluating $\log 0$ when the sigmoid saturates.

An important property: BCE is a *proper scoring rule.* If the true probability is p, the expected loss $-[p \log q + (1 - p) \log(1 - q)]$ is minimised exactly at $q = p$. The probabilities produced by the model are therefore calibrated, which is exactly what the simulation needs.

6.2 Huber loss

For a residual $r = \hat{y} - y$ and a threshold δ (here $\delta = 1$):

$$\text{Huber}_\delta(r) = \begin{cases} \frac{1}{2} r^2 & |r| \le \delta, \\ \delta\big(|r| - \frac{1}{2}\delta\big) & |r| > \delta. \end{cases} \tag{22}$$

It is quadratic near zero (smooth, with a gradient proportional to the error) and linear far from it (the gradient is bounded by δ). Daily listening has atypical days; with MSE a single extreme day would dominate the gradient, while with Huber its influence is limited. Since the values are scaled (standard deviation close to 1), $\delta = 1$ separates "normal" errors from "large" ones.

6.3 Masked averages

Let $m_{i,h} \in \{0, 1\}$ be a mask (for example, $m = \texttt{active}$). A masked term is the average over the valid cells of the batch only:

$$\mathcal{L}_{\text{mask}} = \frac{\sum_{i,h,k} m_{i,h}\, \ell_{i,h,k}}{\sum_{i,h,k} m_{i,h}}. \tag{23}$$

If a batch has no valid cell (for example, no payment), the term is zero.

6.4 The five terms

$$\begin{aligned} \mathcal{L} = w_{\text{act}} \underbrace{\text{BCE(active)}}_{\text{every day}} + w_{\text{lis}} \underbrace{\text{Huber(listening)}}_{\text{active days}} + w_{\text{pay}} \underbrace{\text{BCE(pay)}}_{\text{every day}} \\ + w_{\text{amt}} \underbrace{\text{Huber(amount, plan)}}_{\text{payment days}} + w_{\text{flag}} \underbrace{\text{BCE(cancel, renew)}}_{\text{payment days}}. \end{aligned} \tag{24}$$

6.5 Fixed weights from a constant predictor

The five terms have very different scales. The BCE of a rare event such as a payment (about 3% of days) is small even if the model learns nothing, while the Huber loss of listening can be much larger. Summed without weights, the gradient would be dominated by the largest term.

The solution is to divide each term by the loss of a *constant predictor*, which ignores the input:

$$w_j = \frac{1}{\mathcal{L}_j(\text{constant}^*)}. \tag{25}$$

- For a binary target, the best constant is the prevalence $\bar{y}$, and its loss is the binary entropy $-[\bar{y} \log \bar{y} + (1 - \bar{y}) \log(1 - \bar{y})]$.
- For a quantity, the constant is its mean over the valid days.

With these weights, each term starts near 1 and its value reads as "fraction of the naive predictor's loss". A term at 0.6 means the model reduced that loss by 40%.

The weights are computed *once*, on a sample of `fit`, and shared by LSTM and GRU. They are not searched by Optuna: if the weights were hyper-parameters, the search could "improve" the objective simply by lowering the weight of the hard term.

6.6 Why there is no `pos_weight` on payments

In imbalanced classification it is common to multiply the loss of the positives by a factor $w > 1$. With that change, the probability that minimises the expected loss is no longer p but

$$q^* = \frac{w\,p}{w\,p + (1-p)}. \tag{26}$$

For example, with $p = 0.03$ and $w = 10$, $q^* = 0.30/(0.30 + 0.97) \approx 0.24$: the model would say the user pays with a 24% probability when the real one is 3%. In simulation, almost every simulated user would pay every week. Since the simulator samples from these probabilities, they must stay calibrated, so `pos_weight` is not used.

7 Training and hyper-parameter search

7.1 Backpropagation through time

The gradient of $\mathcal{L}$ enters through the head, passes through BatchNorm to h_{83} and travels back through the 83 days via the recurrent connections. Since the recurrent weights are shared across days, their gradient is the sum of the contributions of every step:

$$\frac{\partial \mathcal{L}}{\partial \theta_{\text{RNN}}} = \sum_{t=1}^{83} \frac{\partial \mathcal{L}}{\partial h_t}\,\frac{\partial h_t}{\partial \theta_{\text{RNN}}}. \tag{27}$$

The difference with the classifier is at the start: the error signal is no longer a single $\hat{p} - y$ but the sum of the gradients of the 91 targets, each with its own weight and mask.

7.2 Optimisation

- **AdamW**: an adaptive step per weight and decoupled weight decay, which shrinks the weights towards zero at every step.
- **Gradient clipping** at norm 1.0, to avoid the exploding gradients typical of recurrent networks.
- **Mixed precision** on GPU: the forward pass runs in `float16`; the loss is computed in `float32`.
- **Early stopping**: after every epoch the loss is measured on `stop`; if it does not improve for 5 epochs, training stops and the weights of the best epoch are restored.

7.3 Partitions

Partition	Cohort	Use
`fit` (75%)	February	training; computing the loss weights
`stop` (12.5%)	February	early stopping
`tune` (12.5%)	February	Optuna objective
`development`	all of February	final refit
`test`	March	final evaluation, once

Table 3: The same partitions as the classifier.

7.4 Optuna

For each cell, Optuna uses the TPE sampler (*Tree-structured Parzen Estimator*) with the same seed, 10 trials and the same search space:

Hyper-parameter	Space
layers	$\{2, 3\}$
H_d	$\{64, 128, 256\}$
learning rate η	$[10^{-4}, 10^{-2}]$, log scale
weight decay λ	$[10^{-6}, 10^{-2}]$, log scale
batch size	512

TPE models separately the density of the hyper-parameters that gave good results, $\ell(x)$, and of those that gave bad ones, $g(x)$, and proposes the next point where the ratio $\ell(x)/g(x)$ is largest. The objective is the weighted composite loss on `tune`. Unlike the classifier, there is no threshold search: the simulation uses the probabilities directly.

Finally, each winning architecture is refitted on all of February for its best number of epochs and predicts March once. The checkpoint stores the weights, hyper-parameters, loss weights, window sizes and scaler phase: everything needed to reload it in the environment.

8 Simulation: from probabilities to one possible week

8.1 Sampling, not thresholding

For each user and day h, the network gives $\hat{\pi}_h = P(\text{active})$ and $\hat{\rho}_h = P(\text{pay})$. A simulated week is obtained by sampling:

$$A_h \sim \text{Bernoulli}(\hat{\pi}_h), \qquad P_h \sim \text{Bernoulli}(\hat{\rho}_h), \qquad C_h, R_h \sim \text{Bernoulli}(\cdot) \text{ if } P_h = 1. \tag{28}$$

Why not a threshold of 0.5? Suppose the network gives a user $\hat{\rho}_h = 0.2$ on each of 3 days around their expiry. No day exceeds 0.5, so with a threshold the user would *never* pay. Yet the probability of paying at some point during those three days is $1 - 0.8^3 \approx 0.49$. Since the daily

payment probability almost never exceeds 0.5, a threshold would make almost nobody pay in the simulation. Sampling respects the probabilities and also makes the environment stochastic, which is what reinforcement-learning algorithms (PPO, DQN) need.

On active days the predicted amount $\hat{m}_h$ is written; on payment days, the predicted amount and plan. These quantities are written without extra noise, so the simulation has somewhat less spread than reality.

8.2 State reconstruction rules

Let s_h be the number of days since the last payment, e_h the days until expiry and P_h, C_h, R_h the sampled events. Starting from the real state of day 82 (read from the tensor and decoded):

$$\text{no payment } (P_h = 0): \quad s_h = s_{h-1} + 1, \qquad e_h = e_{h-1} - 1, \tag{29}$$

$$\text{payment } (P_h = 1): \quad s_h = 0, \qquad e_h = \begin{cases} 0 & \text{if it cancels } (C_h = 1), \\ \max(e_{h-1} - 1,\, 0) + \text{plan}_h & \text{if it renews.} \end{cases} \tag{30}$$

In addition, on a payment, `tx_last_is_cancel` $= C_h$, `tx_last_is_auto_renew` $= R_h$, `tx_state_known` = `tx_expiry_known` $= 1$ and `tx_state_ambiguous` $= 0$. Finally, the values are encoded again with $\log 1p$ and slog.

The renewal rule reflects that an early renewal *extends* the membership from the previous expiry, whereas if it has already expired, the new membership starts on the payment day:

- Pays on the expiry day: the day before, $e = 1$, so $\max(1 - 1, 0) + 30 = 30$ days.
- Pays 5 days late: the day before, $e = -4$, so $\max(-5, 0) + 30 = 30$ days from today.

Expiry approximation. The export uses the real expiry date of each transaction; the simulator does not know it and derives it from the predicted plan days. To measure how much error this rule alone introduces, the state is rebuilt from the *real* events (source `oracle_events` in the evaluation). If that error is small, the simulator's error comes from the predicted events, not from the rule.

8.3 Chaining weeks

To simulate more than one week, each sampled week (already in the export encoding) is appended to the end of the window and the 7 oldest days are dropped:

$$W^{(k+1)} = \left[\, W^{(k)}_{8:83} \;\middle|\; \text{week}^{(k)} \,\right], \qquad W^{(k)} \in \mathbb{R}^{83\times 20}. \tag{31}$$

The new window is scaled and passed through the network again. Because the states were rebuilt with the export's rules, the input stays coherent week after week.

8.4 Use from the agent's environment

1. The environment applies an action (for example, an accepted discount adds a transaction).
2. The user's last 83 days are built in the export encoding.
3. `simulate_next_week` returns one sampled week; `rollout`, several.
4. The environment reads what it needs from the simulated week (activity, payments, expiry) to compute the reward or the next state.

The model has no calendar input, so it can be applied to the days after the cutoff even though it was trained to predict days 83–89. The implicit assumption is that behaviour is stable over time.

9 Evaluation

9.1 Principle: compare with naive predictors

An MAE of 0.8 says nothing on its own. Each group of targets is compared with predictors that learn nothing. If the model does not beat them, it learned nothing useful for that group, which is also a valid result.

9.2 Listening

Point forecast. $\hat{y}_{i,h,k} = \hat{\pi}_{i,h}\,\hat{m}_{i,h,k}$, undoing the scaling to return to $\log 1p$.

Naive predictors.

- **Persistence:** every future day repeats day 82.
- **7-day mean:** every future day is the mean of days 76–82.
- **Previous week:** day $83 + h$ copies day $76 + h$ (same weekday).

Metrics.

$$\mathrm{MAE} = \frac{1}{NHK}\sum_{i,h,k}|\hat{y}_{i,h,k} - y_{i,h,k}|, \qquad \mathrm{RMSE} = \sqrt{\frac{1}{NHK}\sum_{i,h,k}(\hat{y}_{i,h,k} - y_{i,h,k})^2}, \tag{32}$$

$$\mathrm{skill} = 1 - \frac{\mathrm{MAE}_{\mathrm{model}}}{\mathrm{MAE}_{\mathrm{persistence}}}. \tag{33}$$

A skill of 0.15 means 15% less error than repeating the last day; a negative value means the model is worse than doing nothing.

Breakdowns.

- **By horizon day** ($h = 1, \ldots, 7$): the error should grow with h. If it is flat, the model is predicting something close to a constant.
- **By channel.**
- **Active versus inactive days**, and users with at least one listening record only. Many users have no logs at all, and a model that predicts almost zero can have a low overall MAE while being useless.

A note on MAE and the point forecast. $\hat{\pi}\,\hat{m}$ approximates the conditional *mean*, whereas the MAE is minimised by the *median*. When $\hat{\pi} < 0.5$, the true median is 0. This is why a naive predictor such as persistence can have a low MAE on inactive days, and another reason to look at the active/inactive breakdown and the event metrics, not only the overall MAE.

9.3 Events: activity and payment

ROC-AUC. The probability that a randomly chosen positive day scores higher than a randomly chosen negative day. It does not depend on prevalence.

AP (average precision). The area under the precision–recall curve. It is more informative than the AUC for rare events such as payments: a random predictor has AP $\approx$ prevalence.

Brier. $\frac{1}{n}\sum(\hat{p} - y)^2$. It measures calibration and discrimination at once; it matters because the simulation uses the probabilities.

Naive predictors for `active`: activity on day 82 and the share of active days between day 76 and day 82.

Expiry rule for `pay`. A user with auto-renewal, no cancellation and a known expiry usually pays when the membership expires. If on day 82 they have e days left, the score for day h is

$$\text{score}_h = \mathbf{1}[\text{eligible}]\; e^{-|e-h|}, \tag{34}$$

which peaks (= 1) on the expiry day and decreases on both sides, tolerating payments one or two days early or late. It is a strong naive predictor: if the network does not beat it, it learned nothing about payments. An honest result may be that the network helps for users without auto-renewal (irregular payments) and not for the rest.

Payment within the week. The probability that the user pays at least once in the 7 days is

$$P(\text{pays during the week}) = 1 - \prod_{h=1}^{7}(1 - \hat{\rho}_h), \tag{35}$$

assuming independence across days. The assumption is approximate (two payments in the same week are rare) but useful for the weekly ranking.

9.4 Amount and plan

They are measured on real payment days only, undoing the encoding to express them in NTD and days. The naive predictor is *the last amount paid* in the input window (or the median, if the user did not pay in the window).

9.5 Realism of the simulation

One week per user is sampled and compared with the real March week:

- mean share of active days,
- share of users who pay during the week,
- distribution (quartiles and histogram) of `total_secs` on active days.

A simulator with a good MAE but unrealistic users (everyone active every day, or everyone paying) would be useless for the agent.

9.6 Error of the rebuilt state

The real state on days 83–89 is compared with three sources:

Source	What it measures
`no_payment_rule`	nobody pays; the counters only advance. Lower reference.
`oracle_events`	the *real* events go through the reconstruction. Error due to the expiry rule alone.
LSTM / GRU	events sampled from the model. Total error of the simulator.

Metrics: MAE in days of `days_to_expire` and `days_since_last`, and accuracy of the auto-renewal flag.

A caveat: because the simulator *samples*, when it gets the week of the payment right but not the exact day, the error in days is large on that day. Its state MAE can therefore be higher than that of `no_payment_rule`, even though every trajectory is coherent. What matters for the agent is that the distribution of payments is realistic, and that is what the realism table measures.

9.7 LSTM versus GRU: paired bootstrap

For each user the listening MAE is computed with both models, a_i and b_i. The quantity of interest is $\bar{d} = \frac{1}{N}\sum_i (a_i - b_i)$. In each bootstrap replicate, users are resampled with replacement, *the same users for both models*, and $\bar{d}$ is recomputed:

$$\bar{d}^{(b)} = \frac{1}{N}\sum_i w_i^{(b)}\,(a_i - b_i), \qquad w^{(b)} \sim \text{Multinomial}\big(N;\, \tfrac{1}{N}, \ldots, \tfrac{1}{N}\big). \tag{36}$$

The 95% interval is given by the 2.5 and 97.5 percentiles of the replicates. Pairing is essential: both models make mistakes on the same hard users, and that correlation greatly reduces the variance of the difference. If the interval contains 0, there is no reliable difference and the smaller, faster model (usually the GRU) is preferable.

With almost a million users, tiny differences can be "significant". The size of the difference, the number of parameters and the time per epoch are therefore reported as well, and the decision is made with all three.

10 Limitations

- **Not comparable with the classifier.** One predicts churn (log-loss, AUC over users) and the other trajectories (MAE, AUC over days). They are different tasks.
- **It does not learn the response to offers.** The training data contain no retention actions. The simulator only projects forward the history an action leaves behind; accepting or rejecting the offer is still the job of the customer agent.
- **Approximate expiry.** It is derived from the predicted plan days; `oracle_events` quantifies that error.
- **Amounts without noise.** The simulated spread is smaller than the real one.
- **Independence across days** when sampling: the events of the week are sampled separately, conditioned on the same input. The network can capture patterns such as "if they pay on Monday they do not pay on Tuesday" only through the marginal probabilities of each day.
- **Temporal stability.** The model is trained on days 83–89 of the window and used after the cutoff; behaviour is assumed not to change between the two periods.